INTERNAL ASSIGNMENT

NAME	**SOHAIL ANSARI**
ROLL NO	**261450205242**
SESSION	**JULY 2026**
PROGRAM	**BACHELOR OF COMPUTER APPLICATIONS (BCA)**
SEMESTER	**I**
COURSE CODE & NAME	**DCA1107 & C Programming**

SET – 1

QUESTION -1: (a) Explain the different data types available in C with suitable examples. (b) Explain formatted input and output operations in C using printf() and scanf().

ANSWER – (a) Explain the different data types available in C with suitable examples.

Data types specify the kind of data a variable can hold, the amount of memory it occupies, and the operations permitted on it.

1. Primary (Fundamental) Data Types
   - int — stores whole numbers; typically 2 or 4 bytes; format specifier %d. Example: int age = 21;
   - char — stores a single character (ASCII value); 1 byte; %c. Example: char grade = 'A';
   - float — single-precision real numbers; 4 bytes; %f. Example: float pi = 3.14;
   - double — double-precision real numbers; 8 bytes; %lf. Example: double salary = 45000.75;
   - void — denotes "no value"; used for functions that return nothing and for generic pointers.

Qualifiers — short, long, signed, unsigned — modify the range and storage of the basic types, e.g. unsigned int, long long int.

```
#include <stdio.h>
int main() {
  int a = 10;
  char c = 'X';
  float f = 2.5f;
  double d = 12345.6789;
  unsigned int u = 4000000000u;
  printf("%d %c %.1f %.4lf %u\n", a, c, f, d, u);
  printf("int=%zu char=%zu float=%zu double=%zu bytes\n",
        sizeof(int), sizeof(char), sizeof(float), sizeof(double));
  return 0;
}
```

2. Derived Data Types — built from fundamental types:
   - Arrays — collection of same-type elements: int marks[5];
   - Pointers — hold addresses: int *p = &a;
   - Functions — have a return type: int add(int, int);
3. User-Defined Data Types
   - struct — groups different types into one unit: struct Student { char name[20]; int roll; };
   - union — like struct but all members share the same memory.

- enum — named integer constants: enum Day { MON, TUE, WED };
- typedef — creates an alias: typedef unsigned long ulong;

Summary: the choice of data type affects memory usage, range, precision, and performance; using sizeof() confirms the size on a given compiler.

(b) Explain formatted input and output operations in C using printf() and scanf().
Formatted I/O means reading and writing data in a specified format, using format specifiers that tell the function how to interpret the data. The declarations are in <stdio.h>.
printf() — Formatted Output
Syntax: printf("format string", variable_list);
The format string contains plain text, escape sequences (\n, \t), and conversion specifiers beginning with %. Each specifier corresponds to one argument, in order.

```
int roll = 101; float pct = 87.456; char sec = 'B';
printf("Roll: %d\n", roll);
printf("Percentage: %.2f\n", pct);   // 87.46
printf("Section: %c | Hex: %x\n", sec, roll);  // Hex: 65
printf("|%6d|%-6d|\n", roll, roll);  // width 6, right/left aligned
```

printf() returns the number of characters printed, which can be used for validation.
scanf() — Formatted Input
Syntax: scanf("format string", &variable_list);
The address-of operator & is essential because scanf() must know where to store the value. It returns the number of items successfully read.

```
int age; float salary; char name[20];
printf("Enter age: ");   scanf("%d", &age);
printf("Enter salary: "); scanf("%f", &salary);
printf("Enter name: ");   scanf("%19s", name);   // no & for arrays
```

Important points: %s stops reading at the first whitespace; width limit (%19s) prevents buffer overflow; mixing scanf() and fgets() can leave a newline in the buffer, so use getchar() to clear it.
Common Format Specifiers

Specifier	Data type	Specifier	Data type
%d, %i	int	%c	char
%f	float	%lf	double
%s	string	%u	unsigned int
%x, %o	hex, octal	%p	pointer
%e	scientific	%%	literal %

Formatting also supports width (%5d), precision (%.2f), flags (- left-justify, + sign, 0 zero-padding), and * for dynamic width. Proper formatted I/O makes program output readable and aligned and input controlled and safe.

QUESTION -2: Explain arithmetic, relational, and bitwise operators in C with suitable examples.

ANSWER – An operator performs an operation on operands; C has a rich set of operators classified by function.

1. Arithmetic Operators

Used for mathematical calculations: + (addition), - (subtraction), * (multiplication), / (division), % (modulus, remainder), and the unary ++/--.

Key points: integer division truncates (7/2 gives 3); % works only on integers; division or modulus by zero is a runtime error.

```
int a = 17, b = 5;
printf("%d %d %d %d %d\n", a+b, a-b, a*b, a/b, a%b);
// 22 12 85 3 2
float x = 17.0f/5;          // 3.400000 (float division)
int i = 5, j = 5;
printf("%d %d %d %d\n", i++, i, ++j, j);  // 5 6 6 6
```

++/-- have pre (increment then use) and post (use then increment) forms — a frequent source of bugs.

2. Relational Operators

Compare two values and return 1 (true) or 0 (false): == (equal), != (not equal), > (greater), < (less), >=, <=.

They are used mainly in conditions of if, while, and for.

```
int a = 10, b = 20;
if (a == b) printf("Equal\n");
else if (a < b) printf("a is smaller\n");   // printed
printf("%d %d\n", a > b, a != b);           // 0 1
```

Common pitfall: writing if (a = b) instead of if (a == b) — assignment returns b, which is treated as true. Write if (10 == a) to let the compiler catch such errors.

3. Bitwise Operators

Operate directly on the individual bits of integer operands: & (AND), | (OR), ^ (XOR), ~ (NOT/complement), << (left shift), >> (right shift).

- a & b — bit is 1 only if both bits are 1 (used for masking).
- a | b — bit is 1 if either bit is 1 (used for setting bits).
- a ^ b — bit is 1 if bits are different (used for toggling and swapping).
- ~a — flips all bits.
- a << n — shifts bits left by n, equivalent to multiplying by 2^n.
- a >> n — shifts bits right, equivalent to dividing by 2^n (for unsigned/positive values).

```
int a = 12, b = 10;         // a = 1100, b = 1010
printf("%d %d %d %d\n", a & b, a | b, a ^ b, ~a);  // 8 14 6 -13
printf("%d %d\n", a << 2, a >> 2);                 // 48 3
int x = 5, y = 7;
x ^= y; y ^= x; x ^= y;     // swap without temp: x=7, y=5
if (a & 1) printf("Odd\n"); else printf("Even\n");  // even check
```

Bitwise operators are widely used in low-level programming, flags, device drivers, and for efficient arithmetic and parity/even-odd checks, where they are much faster than multiplication and division.

Precedence note: arithmetic operators rank above relational, which rank above bitwise &, ^, |; use parentheses for clarity.

QUESTION -3: Explain the switch-case statement in C and compare it with the if-else statement.

ANSWER – switch-case Statement

The switch statement is a multi-way decision construct that transfers control to one of several labelled blocks, based on the value of a single integer expression.
Syntax

```
switch (expression) {
  case constant1:
    statements;
    break;
  case constant2:
    statements;
    break;
  ...
  default:
    statements;
}
```

Rules of the switch statement

1. The expression must evaluate to an integer or character type (int, char, enum) — float and strings are not allowed.
2. Each case label must be a compile-time constant and unique.
3. break exits the switch. Without break, control "falls through" to the next case — sometimes useful, usually a bug.
4. default is optional and executes when no case matches; it can appear anywhere but is conventionally last.
5. Cases may be grouped to share code, and case labels cannot have ranges.

Example — simple calculator

```
#include <stdio.h>
int main() {
  char op; float a, b;
  printf("Enter op (+,-,*,/): "); scanf(" %c", &op);
  printf("Enter two numbers: "); scanf("%f %f", &a, &b);
  switch (op) {
    case '+': printf("Sum = %.2f\n", a + b); break;
    case '-': printf("Diff = %.2f\n", a - b); break;
    case '*': printf("Product = %.2f\n", a * b); break;
    case '/':
      if (b != 0) printf("Quotient = %.2f\n", a / b);
      else printf("Division by zero\n");
      break;
    default: printf("Invalid operator\n");
  }
  return 0;
}
```

Example — grouped cases (vowels)

```
switch (ch) {
  case 'a': case 'e': case 'i': case 'o': case 'u':
    printf("Vowel\n"); break;
  default: printf("Consonant\n");
}
```

Comparison with if-else

Basis	switch-case	if-else
Tested value	Only equality against constants	Any condition — ranges, relational, logical
Expression type	Must be integer/char	Any expression yielding true/false
Case labels	Constant expressions only	Variable conditions allowed
Multiple values	Compact via grouped cases	Repeated == comparisons
Ranges	Not supported (case 1–10 invalid)	Supported: if (n>=1 && n<=10)
Floats/strings	Not allowed	Allowed
Readability	Clearer for many fixed alternatives	Can become deeply nested and messy
Execution speed	Compiler may build a jump table — faster	Sequential evaluation — slower for many branches
Flexibility	Restricted	Highly flexible
Default/else	default label	else clause
Fall-through	Possible without break	Not applicable

Choosing between them: use switch when there are many fixed integer alternatives (menus, calculator, character classification, state machines) — it is more readable and often faster. Use if-else when conditions involve ranges, floating-point values, strings, or compound logical expressions (e.g. if (marks >= 60 && attendance >= 75)).
Both constructs can be nested and combined; a switch may contain an if-else inside a case (as in the division-by-zero check above), and an if-else ladder can always be written as a switch only if all tests are equality against constants.

SET – 2

QUESTION -4: EXPLAIN ONE-DIMENSIONAL ARRAYS IN C WITH THEIR DECLARATION, INITIALIZATION, AND ACCESSING METHODS.

ANSWER - **DEFINITION**

AN **ARRAY** IS A COLLECTION OF A **FIXED NUMBER OF ELEMENTS OF THE SAME DATA TYPE**, STORED IN **CONTIGUOUS MEMORY LOCATIONS** AND ACCESSED USING A **SINGLE NAME PLUS AN INDEX**. A **ONE-DIMENSIONAL (1-D) ARRAY** REPRESENTS A SINGLE LIST OF VALUES — FOR EXAMPLE, MARKS OF STUDENTS OR TEMPERATURES OF DAYS.

DECLARATION

SYNTAX: DATA_TYPE ARRAY_NAME[SIZE];

```
INT MARKS[5];      // 5 INTEGERS, INDICES 0 TO 4
FLOAT PRICE[10];   // 10 FLOATS
CHAR NAME[20];     // 20 CHARACTERS (STRING OF UP TO 19 CHARS + '\0')
```

THE **SIZE MUST BE A POSITIVE INTEGER CONSTANT** IN CLASSIC C (VARIABLE-LENGTH ARRAYS ARE PERMITTED IN C99 BUT SHOULD BE AVOIDED IN GENERAL PRACTICE). MEMORY ALLOCATED = **SIZE × SIZEOF(ELEMENT)** — FOR INT MARKS[5] ON A 4-BYTE INT MACHINE, **20 BYTES**.

INITIALIZATION

AN ARRAY CAN BE INITIALIZED IN SEVERAL WAYS AT THE TIME OF DECLARATION:

```
INT A[5] = {10, 20, 30, 40, 50};  // ALL ELEMENTS GIVEN
INT B[5] = {10, 20};              // REST BECOME 0 → 10,20,0,0,0
INT C[] = {1, 2, 3, 4};           // SIZE INFERRED AS 4
INT D[5] = {0};                   // ALL ELEMENTS ZERO
INT E[5];                         // UNINITIALIZED — GARBAGE VALUES
INT F[3] = {[2] = 9};             // DESIGNATED INITIALIZER (C99)
```

IMPORTANT: UNINITIALIZED **LOCAL** ARRAYS CONTAIN **GARBAGE**; UNINITIALIZED **GLOBAL/STATIC** ARRAYS ARE ZERO.

ACCESSING ELEMENTS

INDIVIDUAL ELEMENTS ARE ACCESSED USING THE **SUBSCRIPT OPERATOR** [], WITH INDICES RANGING FROM **0 TO SIZE−1**. INDEX SIZE OR BEYOND CAUSES **UNDEFINED BEHAVIOUR** (BUFFER OVERFLOW), AND C PERFORMS **NO BOUNDS CHECKING**.

```
A[0] = 99;              // WRITE FIRST ELEMENT
INT X = A[3];           // READ FOURTH ELEMENT
FOR (INT I = 0; I < 5; I++) // TRAVERSE
 PRINTF("%D ", A[I]);
```

COMPLETE EXAMPLE

```
#INCLUDE <STDIO.H>
INT MAIN() {
 INT N, I, SUM = 0, MAX;
 PRINTF("HOW MANY STUDENTS? "); SCANF("%D", &N);
 INT MARKS[N];
 FOR (I = 0; I < N; I++) {
  PRINTF("MARKS OF STUDENT %D: ", I + 1);
  SCANF("%D", &MARKS[I]);
 }
 MAX = MARKS[0];
 FOR (I = 0; I < N; I++) {
  SUM += MARKS[I];
  IF (MARKS[I] > MAX) MAX = MARKS[I];
 }
 PRINTF("TOTAL = %D, AVERAGE = %.2F, HIGHEST = %D\N",
     SUM, (FLOAT)SUM / N, MAX);
 RETURN 0;
}
```

KEY CHARACTERISTICS AND USES

- ELEMENTS ARE STORED **CONTIGUOUSLY**, SO ACCESS BY INDEX IS **O(1)** AND FAST.
- **INSERTION AND DELETION IN THE MIDDLE ARE COSTLY** (ELEMENTS MUST BE SHIFTED).
- ARRAY NAME ACTS AS A **POINTER TO THE FIRST ELEMENT**; PASSING AN ARRAY TO A FUNCTION PASSES ITS BASE ADDRESS, SO THE FUNCTION CAN MODIFY THE ORIGINAL ELEMENTS.
- COMMON USES: **STORING LISTS AND TABLES, SEARCHING (LINEAR/BINARY), SORTING, COMPUTING STATISTICS, AND IMPLEMENTING OTHER DATA STRUCTURES** SUCH AS STACKS, QUEUES, AND MATRICES (2-D ARRAYS).

LIMITATION: SIZE IS **FIXED AT COMPILE TIME** AND THERE IS **NO AUTOMATIC BOUNDS CHECKING**, SO THE PROGRAMMER MUST VALIDATE INDICES CAREFULLY.

QUESTION -5: Explain for, while, and do-while loops with syntax and suitable examples.
ANSWER –Loops execute a block of statements **repeatedly** until a condition becomes false. C provides three loop constructs.

1. for Loop

Best used when the **number of iterations is known** in advance. It combines initialization, condition, and update in a **single line**.

Syntax: for (initialization; condition; update) { statements; }

Flow: initialization executes once → condition is tested → if true, body executes → update executes → condition is retested. The loop ends when the condition becomes false.

```
#include <stdio.h>
int main() {
  int i, sum = 0;
  for (i = 1; i <= 10; i++) sum += i;
  printf("Sum 1..10 = %d\n", sum);        // 55
  for (i = 1; i <= 5; i++)                // nested loop
    for (int j = 1; j <= i; j++) printf("*");
  return 0;
}
```

All three parts are **optional**: for (;;) is an infinite loop; multiple expressions can be combined with commas (for (i=0, j=5; i<j; i++, j--)).

2. while Loop

A **pre-test (entry-controlled)** loop: the condition is checked **before** the body executes, so the body may run **zero times**.

Syntax: while (condition) { statements; }

```
int n, rev = 0;
printf("Enter a number: "); scanf("%d", &n);
while (n != 0) {
  rev = rev * 10 + n % 10;
  n /= 10;
}
printf("Reverse = %d\n", rev);    // 1234 → 4321
```

Initialization must be done **before** the loop and the update **inside** the body — forgetting either causes an infinite loop.

3. do-while Loop

A **post-test (exit-controlled)** loop: the body executes **first**, and the condition is tested afterwards. Hence the body executes **at least once**, even if the condition is false initially. It **must end with a semicolon**.

Syntax: do { statements; } while (condition);

```
int choice;
do {
  printf("1.Add 2.Subtract 3.Exit\nEnter choice: ");
  scanf("%d", &choice);
  /* process choice */
} while (choice != 3);
```

This is ideal for **menus and input validation**, where the user must be prompted at least once.

Comparison

Basis	for	while	do-while
Type	Entry-controlled	Entry-controlled	**Exit-controlled**
Minimum executions	0	0	**1**
Condition tested	Before body	Before body	**After body**
Best suited for	Known iteration count	Unknown count, condition-driven	Menu, validation, must-run-once
Initialization	Inside for header	Before the loop	Before or inside
Syntax ends with	}	}	} while(cond);
Common use	Counting, arrays, tables	Reading until sentinel	Repeating a menu

Loop control statements: break exits the loop immediately; continue skips the remaining statements of the current iteration and proceeds to the next; goto is available but **discouraged**. All three loops are **interchangeable** — any for can be written as a while and vice versa — but choosing the right one makes the code **clearer and less error-prone**.

QUESTION-6: EXPLAIN HOW FUNCTIONS CAN BE USED WITH DECISION-MAKING STATEMENTS IN C, WITH SUITABLE EXAMPLES.

ANSWER - **Functions and Decision-Making — The Idea**

A **function** is a named, reusable block of code that performs a specific task; **decision-making statements** (if, if-else, else-if ladder, switch) let a program choose among alternatives. Combining them gives **modular, readable, and testable logic**: the decision logic is **encapsulated inside a function**, so the caller simply invokes the function and uses its returned result instead of repeating the conditions everywhere.

Components involved: function **prototype (declaration)**, **definition**, **parameters/arguments**, **return type**, **local scope**, and the **return statement** — which can appear in **different branches** of a decision, so each path returns a different result.

Example 1 — if-else inside a function (even/odd)

```
#include <stdio.h>
```

```
int isEven(int n);              // prototype
int main() {
  int num;
  printf("Enter a number: "); scanf("%d", &num);
  if (isEven(num)) printf("Even\n");   // function used as condition
  else printf("Odd\n");
  return 0;
}
int isEven(int n) {             // definition
  if (n % 2 == 0) return 1;
  else return 0;
}
```

Notice that the function's **return value is itself used as a condition** in main() — the result of one decision drives another.

Example 2 — else-if ladder inside a function (grade calculation)

```
char grade(int m) {
  if (m >= 90) return 'A';
  else if (m >= 75) return 'B';
  else if (m >= 60) return 'C';
  else if (m >= 40) return 'D';
  else return 'F';
}
int main() {
  int marks = 78;
  printf("Grade = %c\n", grade(marks));   // B
  return 0;
}
```

Here the **entire grading policy lives in one function**; if the policy changes, only one place needs editing — a key advantage of modularity.

Example 3 — switch inside a function (menu-driven calculator)

```
float calc(char op, float a, float b) {
  switch (op) {
    case '+': return a + b;
    case '-': return a - b;
    case '*': return a * b;
    case '/': return (b != 0) ? a / b : 0;
    default : printf("Invalid op\n"); return 0;
  }
}
int main() {
  printf("%.2f\n", calc('*', 6, 7));      // 42.00
  return 0;
}
```

Example 4 — passing decision results between functions (leap year)

```
int isLeap(int y) {
  return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
}
int daysInMonth(int m, int y) {
  if (m == 2) return isLeap(y) ? 29 : 28;
  if (m == 4 || m == 6 || m == 9 || m == 11) return 30;
```

```
  return 31;
}
```

This shows **function composition**: the decision in daysInMonth() **calls** another decision function, isLeap().

Advantages of Combining Functions with Decisions

- **Reusability** — the same decision logic is written once and called many times.
- **Readability** — if (isEven(n)) reads better than an inline modulus check.
- **Maintainability and testability** — each function can be unit-tested independently.
- **Abstraction** — the caller need not know *how* the decision is made, only *what* it returns.
- **Avoids repetition** and reduces the chance of inconsistent logic.

Points to remember: functions returning a value must **return on every possible path** (a missing return in one branch gives undefined behaviour); prototypes must **match** the definition; and **pass by value** means the function works on copies — use pointers if the caller's variables must be modified. Well-designed programs therefore combine **functions for structure** with **decision statements for logic**.